Ders 4 Genişletilmiş Notu: Öğrenci Sunumları: CQRS, API Gateway ve Mikroservislerde Test Stratejileri

Ders 4 Genişletilmiş Notu: Öğrenci Sunumları: CQRS, API Gateway ve Mikroservislerde Test Stratejileri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. CQRS: motivasyon ve sorgu gerçekleştirmenin zorlukları
- 2. CQRS'in yapısı ve sorgu servisleri
- 3. CQRS'in avantajları ve dezavantajları
- 4. API Gateway deseni: problem ve tanım
- 5. API Gateway'in görevleri
- 6. API Gateway mimarisi, sahiplik modeli ve BFF
- 7. API Gateway tasarımı, avantajları ve dezavantajları
- 8. API Gateway'in gerçekleştirilmesi: monolitten mikroservise
- 9. Hazır API Gateway ürünleri ve kendi gateway'ini yazmak
- 10. Request routing, API composition ve edge function örnekleri
- 11. Mikroservislerde test: tanım, önem ve test çerçeveleri
- 12. Test ikizleri: stub ve mock
- 13. Test piramidi ve deployment pipeline
- 14. Test türleri
- 15. Bir servis için birim testi: solitary ve sociable testler
- 16. AAA deseni ve UserService örneği
- 17. Entegrasyon testi: tanım ve Order Service'in dört etkileşimi
- 18. Persistence entegrasyon testi
- 19. REST request/response için sözleşme testleri ve Spring Cloud Contract
- 20. Publish/subscribe ve asenkron request/response testleri

Bu ders altı öğrenci sunumundan oluşur. Dersin başında hoca planı açıklar: önce CQRS deseniyle ilgili iki sunum, sonra API Gateway ile ilgili iki sunum, son olarak mikroservislerde test stratejileriyle ilgili üç sunum. CQRS'in tasarımıyla ilgili ikinci sunum bu derste yapılmaz ve sonraki haftaya kalır; böylece derste altı sunum yapılır. Birinci sunum CQRS'i (Bölüm 1-3), ikinci sunum API Gateway desenini (Bölüm 4-7), üçüncü sunum API Gateway'in gerçekleştirilmesini (Bölüm 8-10), dördüncü sunum mikroservislerde test stratejilerini (Bölüm 11-14), beşinci sunum bir servis için birim testi yazmayı (Bölüm 15-16), altıncı sunum entegrasyon testi yazmayı (Bölüm 17-20) anlatır. Sunan kişiler bu notta "bir öğrenci" diye anılır. Hoca bu derste sunumları yönetmiş, akademik bir ek açıklama yapmamıştır.

> **Ek bilgi (derste anlatılmadı):** Sunumlardaki şekiller ve kod listeleri (FTGO, `findOrderHistory()`, `OrderServiceProxy`, Listing 10.1-10.5) slaytlarda kaynak gösterilen Chris Richardson'ın *Microservices Patterns* kitabından alınmıştır. Bu kitapta CQRS büyük olasılıkla yedinci bölümün, API Gateway sekizinci bölümün, test stratejileri dokuzuncu ve onuncu bölümlerin konusudur.

1. CQRS: motivasyon ve sorgu gerçekleştirmenin zorlukları

Birinci sunum CQRS (Command Query Responsibility Segregation) desenini anlatır. Öğrenci sunumu giriş, motivasyon, desene genel bakış, yararlar ve dezavantajlar olarak düzenler; slaytın kenarındaki bölüm adları da bunlardır: INTRO, MOTIVATION, OVERVIEW, BENEFITS, DRAWBACK (ekrandan, 00:10:00).

1.1 Mevcut yaklaşım: API composition

Öğrenciye göre birçok uygulama veriyi ilişkisel bir veritabanında tutar ve mikroservis mimarisinde birden çok servise yayılan sorgular için API composition desenini kullanır. Bu desende sorgu, ilgili servislerin her birine yapılır ve gelen sonuçlar birleştirilir. Ancak bazı sorgular birçok ilişkinin birleştirilmesini gerektirir ve bu, veritabanına ek yük getirir. Bu tür karmaşık sorguların yürütülmesi zamanla sorunlu hale gelir; tek bir veritabanının kapasitesi yetersiz kalabilir. Öğrencinin vurgusuyla, geliştirme süreci ürünü geliştirmekten çok sorguları düzenlemeye ve karmaşık sorgu problemlerini çözmeye kayar.

1.2 `findOrderHistory()` örneği

İlk örnek, bir müşterinin sipariş geçmişini döndüren `findOrderHistory()` sorgu işlemidir. Müşteri bilinir; o müşteriye ait siparişler sipariş tarihine göre sıralanarak listelenmelidir. Dönen her kaydın sipariş numarası, sipariş durumu, toplam tutar ve teslimat zamanı bilgilerini içermesi gerekir.

Bu bilgiyi toplamak için birden çok servise gidilir. Önce Accounting Service'ten ödemeler listelenir. Delivery Service'ten teslimatlar alınıp sıralanır ve ödeme servisinin yanıtıyla birleştirilerek toplam tutar çekilir. Son olarak sipariş numarası ve sipariş durumu için Order Service ile de join yapılır. Alt servislerin bu şekilde join'lerle ilerlemesi süreci zorlaştırır.

Bir zorluk daha vardır: Order ve Kitchen servisleri müşteri kimliği ve menü öğesinin adı gibi alanlarla sorgulanabilirken Delivery ve Accounting servisleri yalnız müşteri kimliğiyle (consumer ID) sorgulanabilir. Servisler arasındaki ilişkiler farklı anahtarlar üzerinden kurulmak zorunda kalır; bu da ek maliyet ve karmaşıklık demektir.

Öğrenci iki çözüm yolunu ve sorunlarını anlatır:

- In-memory join: servislerin cevapları tamamen çekilir ve bütün kayıtlar bellekte birleştirilir. Bu, büyük bir bellek alanı ister ve verimsizdir.
- ID ile sorgulama: bir servisten alınan kimliklerle Order ve Kitchen servisleri tek tek sorgulanır. Filtreleme her seferinde yeniden yapılır ve ek yük getirir. Bu yöntemin verimli olabilmesi için diğer servislerin toplu veri alma (bulk fetch) API'si sunması gerekir; aksi halde tek tek yapılan istekler ağ trafiğini artırır ve performansı düşürür.

1.3 `findAvailableRestaurants()` örneği

İkinci örnek `findAvailableRestaurants()` sorgu işlemidir: belli bir adrese belli bir zamanda teslimat yapabilecek restoranları bulmak. Bu sorgunun temelinde, teslimat adresine belli bir mesafedeki restoranları bulan bir coğrafi konum (geospatial) araması vardır. Zorluk, kullanılan veritabanının bunu desteklemeyebilmesidir. Bazı veritabanları "iki nokta arasındaki mesafeyi bul" ya da "bana en yakın noktadaki restoranları listele" gibi sorguları ve bunlara ait indekslemeyi kolayca destekler; diğerleri desteklemeyebilir.

1.4 Komut ile sorgunun aynı yerde olmasının sorunu

Öğrenci bu örnek üzerinden sorgu ile komutun ilişkisini de anlatır. Restoran verisi Restaurant Service tarafından yönetilir. Bir tarafta sorgular, diğer tarafta bu verileri değiştiren komutlar işlenir. Öğrencinin örneğinde bir kullanıcı hesabını kapattığında, o kişinin restoranda yaptığı alışverişlerin veritabanından silinmesi gerekir. Bu komut işlenirken aynı veritabanına başka bir sorgu yapılıyor olabilir. Bu işlemlerin asenkron gerçekleşmesi veri tutarsızlığına yol açabilir. Öğrencinin vardığı sonuç, sorgu ve komut taraflarının veritabanında ayrılması gerektiğidir. CQRS tam olarak bunu yapar.

2. CQRS'in yapısı ve sorgu servisleri

2.1 Tanım

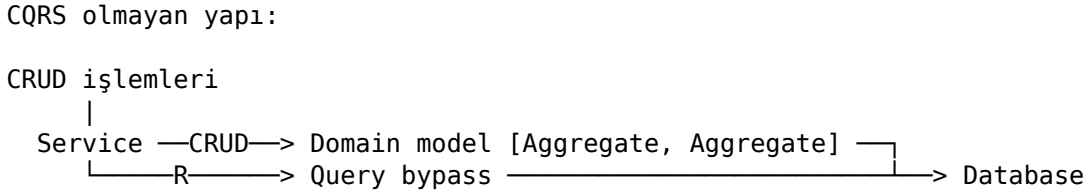
Slayttaki tanım (ekrandan, 00:10:00): CQRS, kalıcı bir veri modelini ve onu kullanan modülleri ikiye ayırır: command side ve query side.

- Command side: create, update ve delete (CUD) işlemlerini uygular. Veride değişiklik olduğunda domain event'leri yayınlar.
- Query side: sorgular (örneğin HTTP GET istekleri) için bir veri modeli sağlar. Verisini, komut tarafının yayınladığı event'lere abone olarak senkronize eder.

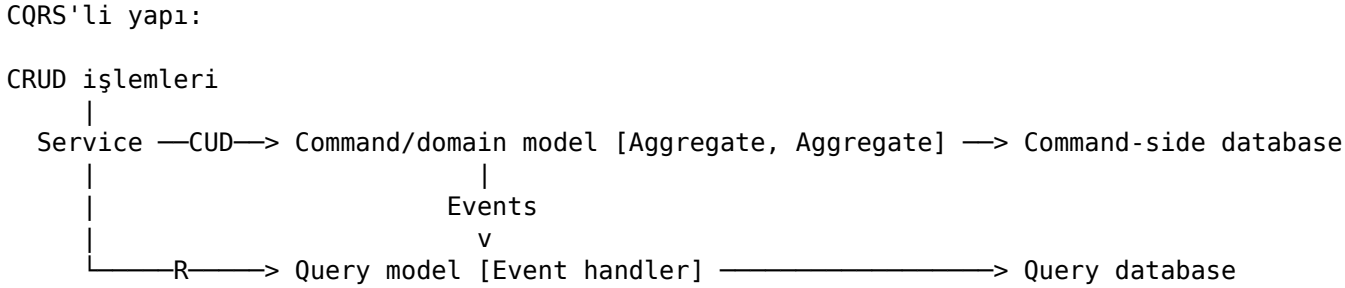
CQRS deseni komut ve sorgu işlemlerini ayrı tutarak her iki tarafın da kendi ihtiyacına göre optimize edilmesini sağlar.

2.2 CQRS olmayan ve CQRS'li yapı

Slayttaki şekil iki yapıyı karşılaştırır (ekrandan, 00:10:00, kitabın 230. sayfasından). CQRS olmayan yapıda servis CRUD işlemlerini alır; hem domain model (aggregate'lar) hem de "query bypass" aynı veritabanını kullanır. Bütün CRUD işlemleri için tek veritabanı vardır.



CQRS'li yapıda servis yine CRUD işlemlerini alır, ama CUD işlemleri command/domain model'e, okuma (R) işlemleri query model'e gider. Command/domain model kendi "command-side database"ine yazar ve event yayınlar. Query model içindeki event handler bu event'leri alır ve ayrı "query database"i günceller. Slayttaki açıklama: oluşturma, güncelleme ve silme için bir veritabanı; sorgular için ayrı bir veritabanı vardır ve sorgu veritabanı, komut tarafı veritabanı her değiştiğinde yayınlanan event'lerle güncel tutulur.

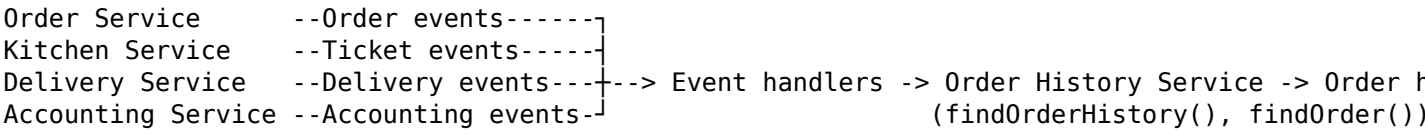


Öğrenci normal yaklaşımı şöyle özetler: aggregate'lar değişiklikleri yapar, veritabanı güncellenir ve gelen sorgu aynı veritabanına yönlendirilir. CQRS'te ise her veri yazma işleminden sonra aggregate'lar event üretir; bu event'ler query model'deki event handler'da yakalanır ve query database güncellenir. Yoğun yazma işlemleri command-side database'e yapılır. Query ve command tarafları ayrıldığı için iki tür işlem birbirinden ayrılır ve sorgu tarafının güncellenmesi asenkron yapılabilir.

2.3 Sorgu servisi: Order History Service

CQRS yalnız bir servisin içinde uygulanmakla kalmaz; ayrı sorgu servisleri (query service) tanımlamak için de kullanılabilir (ekrandan, 00:11:30, kitabın 233. sayfasından). Query service yalnız sorgu işlemlerinden oluşan bir API'ye sahiptir, komut işlemi içermez; veritabanını sorgulayarak sorgu işlemlerini gerçekleştirir.

Örnekte Order History Service `findOrderHistory()` ve `findOrder()` sorgu işlemlerini sunar. Order, Kitchen, Delivery ve Accounting servisleri sırasıyla Order event'leri, Ticket event'leri, Delivery event'leri ve Accounting event'leri yayınlar. Order History Service'in event handler'ları bu event'lere abone olur ve "Order history view database"i (sipariş geçmişi görünüm veritabanı) günceller.



Öğrencinin vurgusu: servisler event yayınlar, Order History Service bunları kendi handler'larında yakalar ve kendi veritabanını günceller. Böylece servislerin asenkron yönetimi de sağlanmış olur.

3. CQRS'in avantajları ve dezavantajları

3.1 Avantajlar

Birinci avantaj, mikroservis mimarisinde sorguların verimli gerçekleştirilmesidir. API composition yaklaşımında büyük veri kümelerinin bellekte birleştirilmesi pahalı ve verimsiz işlemlere yol açabilir. CQRS, önceden hazırlanmış bir görünümü sorgulayarak bunu daha yönetilebilir ve daha verimli kılar.

İkinci avantaj, farklı sorgu türlerinin verimli desteklenmesidir. Bütün sorguları tek bir veritabanı modeliyle desteklemeye çalışmak zordur; bazen bir veritabanının sorgu yetenekleri oldukça sınırlıdır. CQRS birden fazla görünüm tanımlayarak tek veritabanının sınırlamalarından kaçınır; her görünüm belli sorguları etkili biçimde destekleyecek şekilde tasarlanır.

Üçüncü avantaj, ayrı sorumluluklardır (separation of concerns). CQRS bir servisin komut ve sorgu tarafları için ayrı kod modelleri ve ayrı veritabanı şemaları tanımlar. Sorumluluklar ayrıldığı için iki taraf da daha basit ve bakımı kolay olur. Farklı veritabanları kullanıldığı için veritabanlarının yükü de azalır.

3.2 Dezavantajlar

Birinci dezavantaj daha karmaşık bir mimaridir. Geliştiricilerin görünümleri güncelleyen ve sorgulayan sorgu tarafı servislerini yazması gerekir. Ek veri depolarının yönetimi operasyonel karmaşıklık getirir. Uygulama farklı türde veritabanları kullanıyorsa bu durum hem geliştiriciler hem de operasyon ekipleri için zorluğu artırır.

İkinci dezavantaj, komut ve sorgu tarafları arasındaki replikasyon gecikmesidir (replication lag). Komut tarafı bir event yayınladıktan sonra, bu event'in sorgu tarafında işlenip görünümün güncellenmesine kadar bir gecikme olur. İstemci bir nesneyi güncelleyip hemen ardından sorgularsa önceki sürümü görebilir. Öğrencinin örneğiyle: komutla yazma yapılmıştır, ama ekrandaki etikette hâlâ eski bilgi görünür. Bunu çözmek için komut ve sorgu API'leri istemciye sürüm bilgisi sağlayabilir; istemci bu bilgiyle sorgu tarafının güncel olmadığını anlar. Öğrenciye göre değişikliğin olduğunu bildiren ve görünümün güncellenmesini tetikleyen böyle bir mekanizma bu gecikme sorununu çözebilir.

4. API Gateway deseni: problem ve tanım

İkinci sunum API Gateway desenini anlatır.

4.1 Problem: istemci ile servisin doğrudan iletişimi

Problem, istemci ile servisler arasındaki doğrudan iletişimdir. Bu iletişim fazla sayıda isteğe, protokol yönetimi yüküne ve kullanıcı arayüzü tarafında karmaşıklığa yol açar. Öğrenciye göre kitap çözüm olarak iyi tanımlanmış API tasarımı ve API Gateway, Backends for Frontends gibi mikroservis iletişim desenlerini önerir.

İstemci ile mikroservisler arasındaki doğrudan iletişim küçük uygulamalarda uygun olabilir. Büyük ve karmaşık sistemlerde ise yönetim zorluğu gibi sorunlar yaratır:

- İstemci uygulamasının farklı protokollerle birden çok mikroservise yaptığı çoklu istekleri yönetmesi zorlaşır ve istemci kodu karmaşıklaşır.
- İstemci ile mikroservisler arasında sıkı bir bağ oluşur; bu da bakım zorluklarına yol açar.
- Kimlik doğrulama, yetkilendirme ve izleme gibi işlerin her mikroserviste ayrı ayrı uygulanması verimsizdir.

- Farklı istemci türlerine göre yük (payload) değişiklikleri gibi sorunlar ortaya çıkar.

4.2 Tanım

API Gateway, istemci isteklerini mikroservislere yönlendiren, kimlik doğrulama, yük dengeleme ve izleme gibi görevleri üstlenen bir sistemdir. İstemciler için tek bir giriş noktası (single entry point) sağlar ve iç servisler arasındaki iletişimi kolaylaştırır.

5. API Gateway'in görevleri

5.1 Gateway routing

Gateway routing, istemcilerin tek bir uç nokta üzerinden birden fazla mikroservise erişmesini sağlar (ekrandan, 00:20:00). Slayttaki şekilde istemci yalnız Gateway'e bağlanır; Gateway istekleri Customer, ShoppingCart, Discount ve Order servislerine yönlendirir. Slayttaki maddeler:

- Tek bir uç nokta üzerinden birden fazla mikroservise istek yönlendirme.
- İstemcinin birden fazla mikroservisi kullanması gerektiğinde kullanışlıdır.
- Mikroservis değişiklikleri istemci kodunu etkilemez; yalnız yönlendirme yapılandırmasında değişiklik yapılır.
- Gateway routing, arka plandaki mikroservisleri istemciden soyutlar.

Öğrenciye göre kitapta API Gateway'in API sürüm yönetimini de kolaylaştırdığı belirtilir.

5.2 API composition

API Gateway birden fazla mikroservisi çağırıp sonuçları tek bir yanıtta birleştirebilir. Bu, istemci performansını artırır ve istemci kodunu basitleştirir. Örneğin sipariş ayrıntılarını görüntülemek için istemci birden çok istek yapmak yerine tek istek yapar; gateway gerekli verileri farklı servislerden alır ve istemciye tek bir yanıt olarak döner.

5.3 Protocol translation

API Gateway farklı protokoller arasında dönüşüm yapabilir. Örneğin istemciden gelen HTTP isteğini arka uç serviste gRPC'ye çevirir. Bu sayede uyumluluk sağlanır ve sistemin karmaşıklığı azalır.

5.4 İstemciye özel API

Slayttaki başlık: "API Gateway provides each client with client-specific API." API Gateway farklı istemci ihtiyaçlarına çözüm sunar. Örneğin üçüncü taraf bir uygulama tam sipariş ayrıntısını isterken mobil istemci yalnız bir alt kümeyi talep edebilir. Bir yol, istemcilerin hangi verilerin döndürüleceğini belirleyebilmesidir; daha gelişmiş yaklaşım her istemciye özel bir API sağlamaktır. Bu esneklik sağlar.

5.5 Edge function'lar

Edge function'lar istemci isteklerini hızlı ve verimli işlemek için gateway'de (sistemin "kenarında") çalışan işlevlerdir. Yaygın kullanım alanları:

- Kimlik doğrulama (authentication): istemcinin kimliğini kontrol eder.
- Yetkilendirme (authorization): istemcinin işlem yetkisini doğrular.
- Hız sınırlama (rate limiting): istek sayısını belli bir limitte tutar.
- Önbellekleme (caching): yanıtları önbelleğe alarak yükü azaltır.
- Metrik toplama: API kullanımıyla ilgili verileri toplar.
- İstek loglama: gelen istekleri kaydederek izleme sağlar.

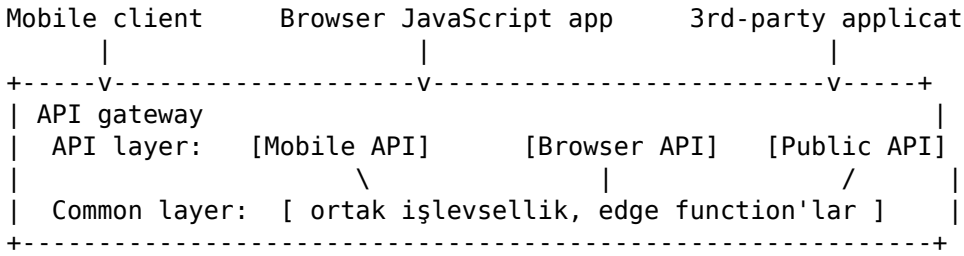
Bu işlevler güvenlik, performans ve izlenebilirlik açısından avantaj sağlar.

6. API Gateway mimarisi, sahiplik modeli ve BFF

6.1 Mimari

Slayttaki mimari şekli (ekrandan, 00:23:40) üç istemciyi gösterir: Mobile client, Browser JavaScript application ve 3rd-party application. Gateway iki katmandan oluşur:

- API katmanı (API layer): bir ya da daha fazla bağımsız API modülünden oluşur. Her API modülü belli bir istemci için bir API uygular: Mobile API, Browser API, Public API.
- Ortak katman (common layer): bütün API'ler için ortak işlevselliği uygular. Kimlik doğrulama, yetkilendirme, hız sınırlama gibi edge function'ları içerir.



API modüllerinin işleyişi kitaba göre şöyledir: yönlendirmede gelen istek API modülü tarafından uygun servis API'sine yönlendirilir; API birleştirmede karmaşık işlemler için birden fazla servis çağrılır ve sonuçlar birleştirilerek istemciye tek yanıt döner. Bunlar yüksek performans ve ölçeklenebilirlik sağlayacak biçimde tasarlanmalıdır.

6.2 Sahiplik modeli

Sahiplik modeli (ownership model), API Gateway'in sorumluluğunun ve yönetiminin ekipler arasında nasıl dağıtılacağını belirler. Merkezi tek bir gateway ekibinin bütün değişiklikleri yapması mikroservislerin bağımsızlığını ve geliştirme hızını engelleyebilir. Kitaptaki modelde ("a client team owns their API module") her istemci ekibi kendi API modülünden sorumludur; kendi API'sini yönetir ve özelleştirir. API Gateway ekibi ise yalnız ortak modülden ve gateway'in operasyonundan sorumludur. Bu model ekiplere API'leri üzerinde daha fazla kontrol verir, hızlı geliştirme döngülerine imkan tanır; ekipler API değişikliklerini bağımsız yapabilir. Öğrenciye göre kitap otomatik dağıtımın, iyi koordinasyonun ve hızlı güncellemelerin bu model için kritik olduğunu söyler.

6.3 Backends for Frontends (BFF)

BFF deseni her istemci türü için ayrı bir API gateway oluşturulmasını önerir. Sorumluluklar netleşir ve ölçeklenebilirlik artar. Ortak işlevsellik paylaşılan bir kütüphane aracılığıyla sağlanır ve bütün gateway'ler aynı teknoloji yığınını kullanır.

Örnek Netflix'tir. Öğrenciye göre Netflix başlangıçta tek bir API kullanmayı denemiş; farklı cihaz türlerinin ihtiyaçlarına yanıt veremeyince her cihaz türü için özel bir API gateway yaklaşımına geçmiştir. BFF modelinde mobil, televizyon gibi her istemci türünün kendi API gateway'i vardır ve her cihaz türüne optimize edilmiş API'ler sağlanır. Slayttaki ek bilgi (ekrandan, 00:28:00): Netflix, mikroservis mimarisini benimseyen öncülerden biri olarak API Gateway üzerinden günde milyonlarca isteği işler; Netflix'in Zuul adlı açık kaynaklı gateway'i dinamik yönlendirme, izleme ve güvenlikten sorumludur.

7. API Gateway tasarımı, avantajları ve dezavantajları

7.1 Tasarım konuları

API Gateway tasarımında dikkate alınacak temel konular:

- Performans ve ölçeklenebilirlik: asenkron (bloke olmayan) G/Ç ölçeklenebilirlik sağlar ama karmaşıktır.
- Bakımı yapılabilir kod: reaktif programlama ile eşzamanlı (concurrent) kod yönetilebilir hale gelir.

- Kısmi hataların ele alınması: bir servisin hatasının bütün sistemi çökertmesini engellemek önemlidir.
- Mimaride iyi bir vatandaş olmak: gateway, servis keşfi ve gözlemlenebilirlik gibi desenleri kullanarak mikroservislerle uyumlu olmalıdır.

7.2 Avantajlar

Slayttaki avantajlar (ekrandan, 00:30:00):

- Basitleştirilmiş API: istemciler için özel API'ler sunar, tek istekle gerekli verilere erişim sağlar, iletişim trafiğini ve istemci kodunu azaltır.
- Esneklik: servislerin ayrıştırılmasını ve değiştirilmesini kolaylaştırır; istemcilerin iç yapıyı bilmesine gerek kalmaz.
- Güvenlik: kimlik doğrulama, yetkilendirme gibi işlevlerle hassas verileri korur.
- Performans ve ölçeklenebilirlik: önbellekleme ve rate limiting gibi mekanizmalarla sistemi daha verimli hale getirir, yüksek trafik altında ölçeklenebilirliği artırır.
- Protokol çevirisi: istemciler ve servisler arasında protokol çevirisi yapabilir.

7.3 Dezavantajlar

- Geliştirme ve yönetim yükü: gateway de geliştirilmesi, dağıtılması ve yönetilmesi gereken bir bileşendir; maliyeti artırır.
- Tek hata noktası riski: bütün istemci istekleri merkezi bir noktada toplandığı için gateway'deki bir arıza sistemin bütününü etkileyebilir.
- Performans gecikmesi: araya giren ek bir ağ adımı vardır.
- Yüksek trafik yönetimi ve karmaşıklık artışı.

Öğrenciye göre kitap bu sorunların BFF deseniyle hafifletilebileceğini önerir. Sonuç olarak API Gateway, mikroservis tabanlı uygulamalarda istemci ile arka uç servisler arasındaki iletişimi yönetir ve güvenlik, önbellekleme gibi kesişen (cross-cutting) işleri üstlenir. Tek bir gateway kullanmak tek hata noktası ve karmaşıklık sorunlarına yol açabilir; öğrencinin önerdiği en iyi uygulama, BFF deseniyle gateway'i her istemci türüne optimize edilmiş daha küçük parçalara bölmektir.

8. API Gateway'in gerçekleştirilmesi: monolitten mikroservise

Üçüncü sunum, önceki sunumun devamı olarak API Gateway'in nasıl gerçekleştirileceğini anlatır. Öğrenci önce mimarileri kısaca karşılaştırır, sonra kitaptaki "hazır ürün mü kullanalım, kendimiz mi yazalım" sorusuna geçer.

8.1 Monolitik mimari

Monolitik mimaride bütün bileşenler sıkı sıkıya bağlı tek bir yapı olarak çalışır. Geliştirmesi basittir, küçük projeler için idealdir; her şey tek birim olarak çalışır ve operasyonel yükü düşüktür. Başta karmaşıklık az görünse de proje büyüdükçe ölçeklenebilirlik sorunları ortaya çıkar, kod karmaşıklaşır. Öğrenciye göre küçük çaplı bir proje yapılmıyorsa ideal değildir. Kitaptaki diyagramda FTGO uygulamasında sipariş ayrıntıları tek bir API çağrısıyla (`getOrderDetails`) monolitik FTGO uygulamasından alınır.

8.2 Mikroservis mimarisi ve istemci sorunu

Mikroservis mimarisinde uygulama küçük, bağımsız ve modüler servislere bölünür. Her servis kendi API çağrılarıyla iletişim kurar ve kendine özgü işlevi yerine getirir. Bağımsız geliştirme geliştirme hızını artırır; ancak öğrenciye göre geliştirme maliyeti düşse de bu modüler yapının bakımı daha yüksek maliyet ister. Ölçeklenebilirlik konusunda monolitten çok daha esnektir ve tek bir servisteki sorun bütün sistemi çökertme tehlikesi yaratmaz. Öte yandan operasyonel yük artabilir.

Asıl sorun istemci tarafındadır. Monolitte tek bir API çağrısıyla alınan bilgi için burada örneğin dört ayrı API çağrısı gerekir. Öğrencinin sorusuyla: istemci dört API için dört farklı URL'i mi bilmek zorundadır, bunlarla nasıl iletişim kuracaktır?

8.3 Çözüm: API Gateway

API Gateway dışarıdan gelen bütün API çağrılarına tek bir giriş noktası sunar. Kullanıcı arka uç servislerine doğrudan maruz kalmaz; yönetim kolaylaşır ve yapı sadeleşir. Gateway şu işleri yapabilir:

- Routing: istemciden gelen istekleri doğru mikroservise yönlendirir.
- Kimlik doğrulama: kullanıcılar arka uç servislerine doğrudan erişmediği için sistem saldırılara karşı bir ölçüde korunur.
- Yük dengeleme: istemci isteklerini dağıtır.
- API aggregation (data aggregation): birden fazla arka uç servisten alınan yanıtları birleştirip tek bir yanıt yapar. Örneğin dört farklı servisten gelen sonuçlar tek bir çıktı olarak döner.
- Rate limiting, izleme (monitoring), trafik takibi ve loglama.
- İstemci ile arka uç arasında protokol çevirisi.

9. Hazır API Gateway ürünleri ve kendi gateway'ini yazmak

9.1 Hazır ürünler

Slayttaki karşılaştırma (ekrandan, 00:40:00):

- AWS API Gateway: "A solid solution for basic API gateway needs but lacks advanced features like API composition." Temel ihtiyaçlar için sağlam bir çözümdür, ama birden fazla arka uç servisin yanıtını birleştirme (API composition) gibi gelişmiş özellikleri yoktur. Öğrenciye göre bu özelliklere ihtiyaç yoksa oldukça başarılıdır; Lambda ile sunucusuz uygulama, DynamoDB ile veritabanı, CloudWatch ile loglama gibi diğer AWS servisleriyle entegre çalışması geliştiriciye çok yardımcı olur.
- AWS Application Load Balancer: "Simple load balancing with limited gateway features." Hafif bir yük dengeleyicidir; gateway özellikleri çok kısıtlıdır, ne edge function ne API composition vardır.
- Kong/Traefik gibi API gateway ürünleri: "Customizable and flexible but requires operational effort." Açık kaynaklı, özelleştirilebilir ve esnektir; ama kurulum, yapılandırma ve bakım için ek zaman ve operasyonel çaba gerektirir.

9.2 Kendi API Gateway'ini geliştirmek

Hazır çözüm yerine kendi API Gateway'imizi yazabiliriz. Öğrenciye göre bu zor değildir; iki kritik nokta vardır: yönlendirme kurallarını doğru tanımlamak ve HTTP proxy davranışını header'larla birlikte doğru biçimde uygulamak.

Öğrenciye göre kitabın yazarı bir gateway çatısı olarak Spring Cloud Gateway'i çok över. Kitaba göre hazır çözümlerin hepsinde bir eksik vardır; AWS'de bile veri birleştirme eksiktir. Spring Cloud Gateway bir çatı olarak alınıp gateway yazıldığında bu sorunlar aşılır: edge function, veri birleştirme ve yönlendirme basit ve temiz biçimde yazılır; reaktif, asenkron ve olay güdümlü (event-driven) yapısı iyi ölçeklenebilirlik sağlar.

10. Request routing, API composition ve edge function örnekleri

Öğrenciye göre bir API Gateway'in dört temel görevi vardır: request routing, API composition (data aggregation), edge function'lar (en temeli kimlik doğrulama) ve protocol translation. Öğrenci ilk üçü için Python örnek kodları gösterir.

10.1 Request routing örneği

Slayttaki tanım: "Request routing forwards client requests to appropriate backend services based on the HTTP method and path." Gateway, istemciden gelen isteği HTTP metoduna ve URL yoluna göre doğrudan arka uca yönlendirir. Gösterilen kod (ekrandan, 00:44:25):

```
@app.get("/orders/{order_id}")
async def get_order(order_id: int, user: dict = Depends(authenticate_token)):
    async with httpx.AsyncClient() as client:
```

```
        order_resp = await client.get(f"{BACKEND_SERVICES['order']}/orders/{order_id}")
        consumer_resp = await client.get(f"{BACKEND_SERVICES['consumer']}/consumers/{order_id}
        delivery_resp = await client.get(f"{BACKEND_SERVICES['delivery']}/deliveries/{order_id
    return {
        "order": order_resp.json(),
        "consumer": consumer_resp.json(),
        "delivery": delivery_resp.json()
    }
```

Öğrencinin anlatımına göre `GET /orders/{order_id}` uç noktası tanımlanmıştır ve bu uç nokta gelen isteği üç farklı arka uç servise yönlendirir.

> **Ders sonrası not:** Öğrenci bu kodu request routing örneği olarak anlattı. Kod Order, Consumer ve Delivery servislerini çağırıp yanıtları tek bir JSON'da birleştirdiği için büyük olasılıkla daha çok API composition örneğidir; saf yönlendirmede istek, metoda ve yola göre tek bir uygun servise iletilir. Ayrıca koddaki consumer ve delivery çağrılarında sipariş kimliğinin (`order_id`) kullanılması bir örnek basitleştirmesidir.

10.2 API composition

Öğrenciye göre data aggregation gerçek bir birleştirme yapar: birden fazla arka uç servisten (Order Service, Consumer Service, Delivery Service) aldığı verileri tek bir JSON yanıtı halinde birleştirip istemciye gönderir. Yukarıdaki kodun `return` kısmı bu birleştirmeyi gösterir.

10.3 Edge function örneği: kimlik doğrulama

Slayttaki tanım: "Edge functions perform tasks like authentication before requests are forwarded to backend services." Gösterilen kod (ekrandan, 00:45:20):

```
def authenticate_token(request: Request):
    token = request.headers.get("Authorization")
    if not token:
        raise HTTPException(status_code=401, detail="Missing Authorization Header")
    try:
        scheme, _, credentials = token.partition(" ")
        if scheme.lower() != "bearer" or not credentials:
            raise HTTPException(status_code=401, detail="Invalid Authorization Header Format")
        payload = jwt.decode(credentials, SECRET_KEY, algorithms=[ALGORITHM])
        return payload
    except JWTError:
        raise HTTPException(status_code=401, detail="Invalid Token")
```

`authenticate_token` fonksiyonu bir istek geldiğinde JWT tabanlı kimlik doğrulama yaparak yetki olup olmadığını kontrol eder. Adımlar:

- `Authorization` başlığı okunur; yoksa 401 ("Missing Authorization Header") döner.
- Başlık değeri ilk boşluktan ikiye ayrılır (`partition(" ")`): şema ve kimlik bilgisi. Şema "bearer" değilse ya da kimlik bilgisi boşsa 401 ("Invalid Authorization Header Format") döner.
- Token gizli anahtar ve algoritmayla çözülür (`jwt.decode`); başarılıysa içerik (payload) döner, `JWTError` olursa 401 ("Invalid Token") döner.

Bu fonksiyon request routing örneğinde `Depends(authenticate_token)` ile uç noktaya bağlanmıştır. Böylece istemciden gelen istekler filtrelenir, yalnız yetkili kullanıcılar arka uç servislerine erişir ve istenmeyen istekler arka uca hiç ulaşmaz.

11. Mikroservislerde test: tanım, önem ve test çerçeveleri

Dördüncü sunum mikroservis mimarisinde test stratejilerini anlatır.

11.1 Test nedir?

Test, bir yazılımın ya da sistemin belli gereksinimlere, beklentilere ve işlevselliğe uygun çalıştığını doğrulama işlemidir. Mikroservis mimarisi, yazılım sistemini küçük, birbirinden bağımsız ve birbiriyle iletişim halinde çalışan servislere böler. Servisler birbirinden farklı olduğu için aralarındaki iletişimi doğrulamak ayrıca önem kazanır.

11.2 Testin önemi

- Hataların erken tespiti: sunulan hizmet kritik olabilir; müşteriye vermeden önce test edip hataları önceden bulmak önemlidir.
- Güvenilir sistem: kullanıcı ve müşteriler için güven ortamı oluşur.
- Sürekli değişim ve hızlı geliştirme: kod değişikliklerinin sistemi bozmadığı ve entegrasyonun sürekli devam ettiği doğrulanır.
- Müşteri memnuniyeti.
- Servisler arası uyum: mikroservisler birçok servisten oluştuğu için aralarındaki uyum da test edilir.
- Maliyet tasarrufu: sistemin düzgün çalışmamasından ve test yapılmamasından doğan ek iş gücü önlenir.

11.3 Otomatik testler ve test çerçeveleri

Testler elle de yapılabilir, otomatik de; öğrenciye göre elle yapılması pek mantıklı değildir. Otomatik testler genellikle bir test çerçevesi (testing framework) kullanılarak yazılır (ekrandan, 00:50:00). Java için JUnit, Python için PyTest, C# için NUnit popüler çerçevelerdir. Test çerçeveleri test yazmayı ve çalıştırmayı birçok açıdan kolaylaştırır:

- Kolay kullanım: testleri düzenlemeyi ve tekrar tekrar çalıştırmayı sağlar.
- Otomasyon: testleri elle çalıştırmak yerine bir komutla bütün testleri çalıştırır.
- Hız ve güvenilirlik: koddaki bir değişikliğin başka bir yerde hata yaratıp yaratmadığını hızla kontrol eder.

11.4 Bir testin yapısı

Slayttaki şekil (ekrandan, 00:50:00) bir test sürecinin nasıl işlediğini gösterir:

```
Test runner --Executes--> Test class [Test method, Test method, Test method]
                                          |
                          Fixture: Setup -> Execute -> Verify -> Teardown  --> SUT
```

- Test runner: testleri çalıştırmaktan sorumlu bileşendir.
- Test class: bir ya da daha fazla test metodu içeren sınıftır; her test metodu bir test senaryosunu temsil eder.
- Fixture: test ortamını hazırlayan ve yöneten süreçtir; dört aşamadan oluşur:
  - Setup: test için gereken ön koşullar hazırlanır (test ortamının kurulumu, veri yükleme, değişkenlerin başlatılması gibi).
  - Execute: test edilen kod parçası (SUT, System Under Test) çalıştırılır.
  - Verify: kodun beklenen sonuçları verip vermediği doğrulanır.
  - Teardown: test sonrası temizlik yapılır (kaynakların serbest bırakılması gibi).

12. Test ikizleri: stub ve mock

Mikroservislerde çok sayıda servis ve dolayısıyla çok sayıda bağımlılık vardır; bu, test süreçlerini karmaşıklaştırabilir ve yavaşlatabilir. Slayttaki örnek (ekrandan, 00:52:35): bir `OrderController` sınıfı `OrderService` gibi başka servisleri çağırıyorsa, bu sınıfı test etmek için bütün sistemi çalıştırmak pratik olmaz. Bu yüzden testlerin izole yapılabilmesi için mock ve stub adı verilen test araçları kullanılır. Bağımlılıkları ortadan kaldırmak için test ikizleri (test doubles) kullanılır: gerçek bağımlılıkların yerine geçip davranışlarını taklit eden nesneler. Test ikizleri SUT'un izole biçimde test edilmesini sağlar.

12.1 Stub

Stub, SUT'un bir bağımlılıktan beklediği sabit değerleri döndürür; bağımlılığı simüle ederek ona olan bağımlılığı ortadan kaldırır. Örneğin `OrderService`'in sipariş oluşturma çağrısına her zaman sahte bir sipariş nesnesi döndürmesi sağlanabilir. Amaç, test edilen servisin dışındaki bütün servisleri simüle ederek yalnız asıl servisi test etmek ve diğer servislerin iş yükünden kurtulmaktır.

Slaytta stub için bir Java örneği de vardır (ekrandan, 00:52:35). `CustomerReader` sınıfı `EntityManager` ile bir müşteriyi bulup ad ve soyadını birleştirerek döndürür:

```
@Entity
public class Customer {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private long id;
    private String firstName;
    private String lastName;
}

public class CustomerReader {
    @PersistenceContext
    private EntityManager entityManager;

    public String findFullName(Long customerID) {
        Customer customer = entityManager.find(Customer.class, customerID);
        return customer.getFirstName() + " " + customer.getLastName();
    }
}
```

Testte `EntityManager` mock'lanır ve `find` çağrısına sabit bir örnek müşteri döndürmesi söylenir; böylece gerçek veritabanı olmadan `findFullName` test edilir:

```
public class CustomerReaderTest {
    @Test
    public void happyPathScenario() {
        Customer sampleCustomer = new Customer();
        sampleCustomer.setFirstName("Susan");
        sampleCustomer.setLastName("Ivanova");

        EntityManager entityManager = mock(EntityManager.class);
        when(entityManager.find(Customer.class, 1L)).thenReturn(sampleCustomer);

        CustomerReader customerReader = new CustomerReader();
        customerReader.setEntityManager(entityManager);

        String fullName = customerReader.findFullName(1L);
        assertEquals("Susan Ivanova", fullName);
    }
}
```

12.2 Mock

Mock, test edilen servisin bağımlılığının doğru biçimde çağrılıp çağrılmadığını doğrular. Öğrenciye göre mock da stub gibi asıl ilgilenilen servisin dışındaki servisleri simüle eder; farkı, ilgili servisin çağrılıp çağrılmadığını doğrulamasıdır.

13. Test piramidi ve deployment pipeline

13.1 Test sınıflandırması

Öğrenciye göre kitapta Brian Marick'in yazılım test süreçlerini daha stratejik ve yapılandırılmış biçimde ele almak için önerdiği bir yaklaşım anlatılır. Bu yaklaşım testlerin neden yapıldığını, hangi hedefe hizmet ettiğini ve hangi test türünün hangi amaca uygun olduğunu sorar. Testlerin önceliklendirilmesi ve hangi test türüne ne zaman odaklanılacağı konusunda rehberlik eder; birim, entegrasyon, performans gibi test türlerinin hedeflerini ve rollerini netleştirir.

> **Ek bilgi (derste anlatılmadı):** Brian Marick'in yaklaşımı genellikle "test quadrant" (test dörtlüsü) olarak bilinir; testleri iş odaklı ya da teknoloji odaklı olmalarına ve programlamayı desteklemelerine ya da uygulamayı eleştirmelerine göre dört bölgeye ayırır.

13.2 Test piramidi

Test piramidi, bir uygulamanın işlevselliğini doğrulamak için hangi test türlerine ne kadar odaklanılması gerektiği konusunda rehberlik eder; her test türünün kapsamı ve karmaşıklığı farklıdır (ekrandan, 00:55:10). Piramidin ana fikri: tabandan yukarı çıktıkça yazılan testlerin sayısı azalmalıdır. Çok sayıda birim testi, az sayıda entegrasyon testi ve en az düzeyde uçtan uca test yazılmalıdır.

```
      /\          End-to-end   : bir uygulamanın kabul testleri        ^ yavaş, kırılgan,
     /  \         Component    : bir servisin kabul testleri           |
    /    \        Integration  : servisin bağımlılıklarıyla iletişimi  |
   /______\       Unit         : iş mantığının testi                   v hızlı, güvenilir
```

- Piramidin temeli, birim testleri: hızlı, basit ve güvenilir oldukları için piramidin temelini oluşturur; geniş kapsamlı test stratejilerinin yapı taşlarıdır. Olabildiğince çok yazılmalıdır.
- Orta katman, entegrasyon ve bileşen testleri: servislerin altyapı (veritabanı, API'ler vb.) ve diğer uygulama servisleriyle etkileşimini test eder. Karmaşıklıkları ve getirdikleri iş yükü birim testlerinden yüksek olduğu için daha az sayıda olmalıdır.
- Piramidin zirvesi, uçtan uca testler: bütün süreci baştan sona, kullanıcı perspektifinden doğrular. Yavaş ve maliyetli oldukları için az sayıda yazılması önerilir.

13.3 Deployment pipeline

Deployment pipeline, kodun geliştiricinin masaüstünden üretim ortamına kadar geçtiği otomatik süreçtir. Yazılım geliştirme ve teslimatını hızlandırırken yüksek kaliteli ve hatasız yazılımın sürekli teslimini sağlar. Aşamaları:

- Commit öncesi testler (pre-commit): geliştirici kodu commit etmeden önce birim testlerini çalıştırır ve kendi yazdığı kodda hata olup olmadığını ilk aşamada görür.
- Commit testleri: kod commit edilir ve sistemle bir bütün halinde çalıştığında sorun olup olmadığı test edilir.
- Entegrasyon testi aşaması: uygulamanın diğer servislerle entegrasyonu doğrulanır.
- Bileşen testi aşaması: servisin bütün işlevselliği kapsamlı biçimde test edilir.
- Dağıtım: bütün testler başarılıysa servis üretim ortamına çıkarılır, yazılım canlıya alınır.

Öğrenciye göre bu, test piramidinin aşamalarına benzer bir süreci otomatik hale getirir; sürekli entegrasyon ve sürekli dağıtım (CI/CD) olarak bilinen yapının aşamaları da aynıdır.

14. Test türleri

14.1 Birim testleri (unit tests)

Birim testleri bir uygulamanın en küçük bileşenlerini, yani fonksiyonlarını ya da metotlarını test eder (ekrandan, 01:00:00). Bağımsız çalıştıkları için yalnız küçük birimler üzerinde yapılır ve genellikle hızlı çalışır. Amaç bir fonksiyonun doğru çalışıp çalışmadığını kontrol etmektir. Slayttaki örnek:

```
public class Calculator {
    public int add(int a, int b) {
```

```
        return a + b;
    }
}

public class CalculatorTest {

    @Test
    public void testAdd() {
        // Test edilen fonksiyon
        Calculator calculator = new Calculator();

        // Fonksiyonu çağır ve sonucu doğrula
        int result = calculator.add(5, 10);

        // Beklenen sonuç 15 olmalı
        assertEquals(15, result);
    }

    @Test
    public void testAddNegativeNumbers() {
        // Negatif sayılarla test yapalım
        Calculator calculator = new Calculator();

        // -5 ve -10'un toplamı -15 olmalı
        int result = calculator.add(-5, -10);

        assertEquals(-15, result);
    }
}
```

Her testte sınıftan bir nesne oluşturulur, ilgili metot çağrılır ve kütüphanenin `assertEquals` fonksiyonuyla çıktının beklenen değere eşit olup olmadığı kontrol edilir: $5+10=15$ ve $(-5)+(-10)=-15$. Öğrenciye göre gereksinimlere ve istenen ayrıntıya göre bunun gibi birçok test senaryosu yazılabilir.

14.2 Entegrasyon testleri (integration tests)

Entegrasyon testleri bir servisin diğer sistemlerle (örneğin veritabanı, API'ler) doğru etkileşimde bulunup bulunmadığını test eder; servislerin birlikte çalışıp çalışmadığı doğrulanır (ekrandan, 01:01:20). Örnekte `OrderService`, `OrderRepository`'ye bağımlıdır:

```
public class OrderService {

    private final OrderRepository orderRepository;

    public OrderService(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }

    public Order createOrder(Order order) {
        return orderRepository.save(order);
    }
}

public class OrderServiceTest {

    @Autowired
    private OrderRepository orderRepository;

    @Autowired
    private OrderService orderService;
```

```
    @Test
    public void testCreateOrder() {
        Order order = new Order("item1", 2);

        Order savedOrder = orderService.createOrder(order);

        Order retrievedOrder = orderRepository.findById(savedOrder.getId()).orElse(null);

        assertNotNull(retrievedOrder);                    // Sipariş kaydedildi mi?
        assertEquals("item1", retrievedOrder.getItem());  // Siparişin adı doğru mu?
        assertEquals(2, retrievedOrder.getQuantity());    // Siparişin miktarı doğru mu?
    }
}
```

Test bir sipariş oluşturur, servis üzerinden kaydeder, sonra repository'nin `findById` metoduyla geri okur ve alanların doğruluğunu kontrol eder. Böylece birbirine bağımlı sınıfların birlikte doğru çalışıp çalışmadığı test edilir.

14.3 Bileşen testleri (component tests)

Bileşen testleri bir servisin izole olarak test edilmesini sağlar; servis bağımsız birimde çalıştırılır, dış bağımlılıklar mock'lanır (ekrandan, 01:03:20). Özellikleri: bir servisin işlevselliğini test eder; bağımlılıklar mock edilir, yani dış servisler taklit edilir. Bir mikroserviste birbirine bağımlı birçok yapı vardır; servisi bir bütün olarak test ederken diğer servisleri gereksiz yere çağırıp onların işlem yüküyle uğraşmak istemeyiz.

```
public class PaymentService {

    private final PaymentGateway paymentGateway;

    public PaymentService(PaymentGateway paymentGateway) {
        this.paymentGateway = paymentGateway;
    }

    public boolean processPayment(Payment payment) {
        // Ödeme işlemine başla
        return paymentGateway.process(payment);
    }
}

public class PaymentServiceTest {

    @Mock
    private PaymentGateway paymentGateway;

    @InjectMocks
    private PaymentService paymentService;

    @Test
    public void testProcessPayment() {
        // Ödeme işlemi için bir ödeme nesnesi oluştur
        Payment payment = new Payment(100, "credit_card");

        // Ödeme gateway'inin başarılı bir ödeme yanıtı döndüğünü belirle
        when(paymentGateway.process(any(Payment.class))).thenReturn(true);

        // Ödeme işlemi başarılı olmalı
        boolean result = paymentService.processPayment(payment);

        assertTrue(result); // Sonuç doğru mu?
```

```
    }
}
```

`PaymentService`, `PaymentGateway`'e bağımlıdır, ama testte bu servisi gerçekten çalıştırmak istemeyiz. Çerçevenin sağladığı yapıyla `process` fonksiyonuna `Payment` türünden bir nesne geldiğinde `true` dönmesi sağlanır. Böylece `PaymentGateway` simüle edilir ve yalnız `PaymentService` sınıfına odaklanılır: `processPayment` normalde gateway'i çağırır, ama burada gateway'in nasıl davranacağı önceden belirlenmiştir.

14.4 Diğer test türleri

- Uçtan uca (end-to-end) testler: bütün sistemin baştan sona test edildiği yöntem.
- Sözleşme (contract) testleri: servisler arasında belli bir sözleşme, belli bir veri formatı vardır. Tüketici (consumer) ve sağlayıcı (provider) tarafları bulunur. Amaç mikroservislerin birbiriyle iletişimini ve API sözleşmelerinin doğruluğunu test etmektir.
- Dayanıklılık (resilience) testleri: hata durumlarında servisin ne kadar dayanıklı olduğunu ve nasıl bir cevap verdiğini görmek için yapılır.
- Performans testleri: mikroservisin yük altında nasıl çalıştığını ve sistemin nasıl tepki verdiğini test eder.
- Güvenlik testleri: yetkisiz erişim ya da başka tehditler karşısında servislerin nasıl tepki verdiğini gösterir.

14.5 Sunumun sonucu

Mikroservislerde test stratejileri yazılımın kalitesini artırmak ve sürdürülebilirliği sağlamak açısından önemlidir. Birim testleriyle her küçük birim hızla test edilir. Test piramidi farklı test seviyelerinin dengeli uygulanması gerektiğini gösterir. Mikroservislerin bağımsız yapısı testlerin paralel çalıştırılmasını mümkün kılar ve test sürelerini büyük ölçüde azaltır. Sözleşme testleri mikroservislerin etkileşimlerini ve uyumunu, dayanıklılık testleri sistemin zorlu durumlara karşı direncini doğrular. CI/CD süreçleri sistemin baştan sona izlemesi gereken yolu belirler. Uçtan uca testler sistemi bir bütün olarak test ederek hem güven hem maliyet açısından avantaj sağlar.

15. Bir servis için birim testi: solitary ve sociable testler

Beşinci sunum bir servis için birim testi yazmayı anlatır. Öğrenci önce birim testi hakkında kısa bilgi verir, sonra bir servis örneğiyle sunumu bitirir.

15.1 Birim testi ve amacı

Birim testi, mikroservislerde ya da monolitik yapıda fark etmeksizin, bir parça için yazılan testtir. Bu parça bir sınıf, bir kontrolcü (controller), bir servis ya da bir entity olabilir. Birim testleri birbirinden bağımsız ve hızlı çalışır; yalnız o birimin işine odaklanarak testin geçip geçmediği hakkında bilgi verir. Bir servis için birim testi yazmanın amaçları:

- Hataların önceden tespit edilmesi.
- Kod kalitesinin artırılması.
- Refactoring'in kolaylaşması: her birim ayrı test edildiği için birimlerin birbirine bağımlılığı azalır.
- Uzun vadede daha düşük maliyet: hatalar önceden yakalanır.

15.2 Solitary ve sociable birim testleri

Birim testleri servislerin bağımlılıklarına göre iki ana başlığa ayrılır: solitary ve sociable.

Solitary birim testinde bağımlılıklar mock'lanır. Slayttaki tanım (ekrandan, 01:10:00): "Testing a unit in complete isolation by replacing its dependencies with mocks, stubs, or fakes to ensure the test focuses solely on the unit's behavior." Bir servis içinde başka sınıflara ait nesneler kullanılabilir; yalnız kendi işine odaklanmak için bu bağımlılıklar mock'lanır ya da

stub'lanır. Böylece diğer bağımlılıkların ne iş yaptığıyla ilgilenmeden yalnız birimin kendi davranışı test edilir. Slayttaki şekilde birim (sınıf, fonksiyon, metot vb.) birim testiyle test edilir; test kurulumu (test set up) bağımlılığı mock'lanmış bir servis olarak hazırlar.

Sociable birim testinde ise bağımlılıklar gerçek haliyle kullanılır. Öğrencinin örneğinde bir servis üç başka servisi tüketir; servisin ne yaptığı bu bağımlılıklara da bağlı olduğu için test bu bağımlılıklarla birlikte ele alınır. Örneğin bir kullanıcı servisi veritabanına sorgu atıyorsa, o sorgu fonksiyonunun ne yaptığı da önemli olur. Kısacası solitary testte bağımlılıklar mock'lanarak birim bağımsızlaştırılır; sociable testte bağımlılıklar sınıf içinde yönetilir ve birim bütün bağımlılıklarıyla birlikte çalıştırılır.

15.3 Hangi sınıf hangi testle?

Kitaptaki şekil (ekrandan, 01:12:20) altıgen mimarideki bir servisi gösterir: dışarıdan `POST /something` ve `GET /something/id` isteklerini alan Controller, mesaj kanallarına bağlı inbound ve outbound mesaj adaptörleri, ortada domain mantığı (Service, Entity, Value object, Saga), Repository ve veritabanı adaptörü. Slayttaki açıklama: "Controller and service classes are often tested using solitary unit tests. Domain objects, such as entities and value objects, are typically tested using sociable unit tests." Şekilde solitary birim testleri Controller'a, Service'e ve mesaj adaptörlerine; sociable birim testleri Entity, Value object ve Saga'ya bağlanır.

Öğrencinin açıklamasına göre solitary birim testi yalnız akışa odaklanan kısımlar için yazılır. Örneğin bir servisin ana görevi vardır; bu görevin dışındaki kısımlar teste dahil edilmeden yalnız işlemin yapılıp yapılmadığına bakılır. Sociable birim testi daha çok entity'ler için yazılır; çünkü entity'ler çok yerde kullanılır ve dışarıya doğru çok bağımlılığı vardır, bu bağımlılıkların doğru çalışıp çalışmadığı kontrol edilmek istenir.

> **Ders sonrası not:** Öğrenci sociable testlerin "entity, value object veya kontroller" için yazıldığını söyledi; slayttaki kitap şekline ve açıklamaya göre controller sınıfları solitary birim testleriyle test edilir. Sociable testler entity ve value object gibi domain nesneleri içindir (şekilde saga da bu gruptadır).

16. AAA deseni ve UserService örneği

16.1 AAA deseni

Birim testi yazılırken AAA deseni kullanılır ve test üç ana kısımdan oluşur:

- Arrange (hazırla): bağımlılıklar yönetilir. Gerekli bağımlılıklar varsa ya mock'lanır ya da, sociable test yazılıyorsa, gerçek halleri sağlanır.
- Act (uygula): test edilen fonksiyon çalıştırılır; bağımlılıklar mock'landıysa onlardan gelecek yanıtların ne olacağı da burada ya da hazırlıkta belirlenir.
- Assert (doğrula): dönen sonucun beklenen sonuçla aynı olup olmadığı ya da bir hata fırlatılması bekleniyorsa bu hatanın yakalanıp yakalanmadığı kontrol edilir.

16.2 Calculator örneği

Öğrenci yalnız test yapısını göstermek için basit bir örnek verir (ekrandan, 01:15:10):

```
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
}

public class CalculatorTest {

    private Calculator calculator; // Unit under test

    @BeforeEach
    void setUp() {
        // Arrange: Initialize the unit under test
```

```
        calculator = new Calculator();
    }

    @Test
    void testAdd_PositiveNumbers() {
        // Act: Call the method being tested
        int result = calculator.add(3, 5);

        // Assert: Verify the outcome
        assertEquals(8, result, "3 + 5 should equal 8");
    }
}
```

Setup kısmında (`@BeforeEach`) test edilen birimin yeni bir nesnesi oluşturulur; bu Arrange adımıdır. `Calculator` sınıfının hiçbir bağımlılığı olmadığı için öğrenci bu testi bir solitary birim testi olarak nitelendirir. Act adımında `add` fonksiyonu çağrılır; toplama sonucu bir tamsayıdır. Beklenen sonuç $3 + 5 = 8$'dir ve Assert adımında bu kontrol edilir.

16.3 UserService için solitary birim testi

Öğrenci asıl konusu için kendi yazdığı bir örneği gösterir (ekrandan, 01:16:50). `UserService`, `UserRepository`'ye bağlıdır; repository veritabanından veri alır. Servisin `getUserName` fonksiyonu bir kullanıcı kimliğiyle kullanıcı adını getirir:

```
@Service
public class UserService {
    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public String getUserName(int userId) {
        User user = userRepository.findUserById(userId);
        return (user != null) ? user.getName() : "User not found";
    }
}
```

Solitary birim testi yazılacağı için `UserRepository`'nin ne yaptığıyla ilgilenmeyiz; repository mock'lanır. Fonksiyonun iki durumu vardır (kullanıcı `null` gelir ya da normal bir kullanıcı gelir); bütün fonksiyonu kapsamak için iki senaryoya iki test yazılır:

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock
    private UserRepository userRepository;

    @Test
    void it_should_get_user_name_when_user_is_null() {
        // given
        Integer userId = 123;
        given(userRepository.findUserById(123)).willReturn(null);
        UserService userService = new UserService(userRepository);

        // when
        String userName = userService.getUserName(123);

        // then
        assertThat(userName).isEqualTo("User not found");
    }
```

```
    @Test
    void it_should_get_user_name() {
        // given
        User user = new User(123, "<ad soyad>");
        Integer userId = 123;
        given(userRepository.findUserById(123)).willReturn(user);
        UserService userService = new UserService(userRepository);

        // when
        String userName = userService.getUserName(123);

        // then
        assertThat(userName).isEqualTo("<ad soyad>");
    }
}
```

Not: slayttaki örnek kullanıcı adı bu notta `<ad soyad>` ile değiştirilmiştir.

Birinci testte repository'ye stub uygulanır: `findUserById(123)` çağrısı `null` döner. When kısmında fonksiyon çalıştırılır; kullanıcı bulunamadığı için dönen değerin "User not found" olduğu Assert ile kontrol edilir. İkinci testte stub'lanmış repository bir `User` nesnesi döndürür; fonksiyonun bu kullanıcının adını döndürdüğü kontrol edilir. Testlerdeki `given / when / then` yorumları AAA yapısının karşılığıdır.

17. Entegrasyon testi: tanım ve Order Service'in dört etkileşimi

Altıncı sunum kitabın entegrasyon testi yazma kısmını anlatır: entegrasyon testinin ne olduğu, niçin kullanıldığı ve farklı etkileşim tipleri için örnekler.

17.1 Tanım ve önem

Entegrasyon testi, test piramidinde birim testinin hemen üstünde, uçtan uca testlerin hemen altında yer alır; birim testlerinden sonra, uçtan uca testlerden önce yapılır. Farklı servislerin ve altyapıların bulunduğu sistemlerde iletişimin doğru kurulup kurulmadığını ve sözleşmelere (contract) bağlı kalınıp kalınmadığını test eder. Önemi:

- Servisler arasındaki iletişimin doğru gerçekleştiği test edilir.
- Beklenmeyen davranışların uçtan uca testlerden önce tespit edilmesi maliyet ve zaman kazandırır.
- Geliştiricilerin doğru geliştirme yapmasına ve diğer geliştiricilerle işbirliğine yardım eder.
- Performans analizleri yapılabilir ve regresyonlar önlenir.

17.2 Order Service'in dört etkileşimi

Kitaptaki şekil (ekrandan, 01:20:00, Figure 10.1) Order Service'in altyapı servisleri ve diğer uygulama servisleriyle etkileşimlerini dört numaralı bölgede gösterir. Şeklin altındaki açıklamaya göre entegrasyon testleri bir servisin istemcileri ve bağımlılıklarıyla iletişim kurabildiğini doğrulamalıdır; ama bütün servisleri test etmek yerine, iletişimi gerçekleştiren tek tek adaptör sınıflarını test etmek stratejidir. Order Service'in doğru çalışması bu etkileşimlerin istendiği gibi çalışmasına bağlıdır; servislerle doğru iletişim kuruluyorsa entegrasyon testi başarıyla geçilmiş olur.

No	Etkileşim tipi	Consumer	Provider
1	REST tabanlı request/response	API Gateway (`OrderServiceProxy`)	Order Service (`OrderController`)
2	Publish/subscribe (event kanalı)	Order History Service (event handler'lar)	Order Service (`OrderDomainEventPublisher`)

No	Etkileşim tipi	Consumer	Provider
3	Kalıcılık (veritabanı)	Order Service (`OrderRepository`)	Veritabanı
4	Asenkron request/response (komut ve yanıt kanalları)	Order Service (`KitchenServiceProxy`)	Kitchen Service (command handler)

Etkileşim 3'te `OrderRepository` adaptör sınıfı Order Service'ten aldığı siparişleri sürekli veritabanına kaydeder. Etkileşim 2'de Order Service, siparişler güncellendikçe ve yeni siparişler geldikçe event yayınlar; bunları Order History Service kullanır. Etkileşim 4'te iletişim komutlar üzerinden yapılır.

17.3 Consumer ve provider tarafı testleri

Entegrasyon testinin önemli bir avantajı, hem consumer hem provider tarafı için iletişimdeki karşı tarafın fiziksel olarak ayağa kaldırılmadan test yapılabilmesidir. Consumer tarafı test edilirken provider'ın ayağa kaldırılmasına gerek yoktur; bu sayede testler çok daha hızlı yapılır. Bu şöyle sağlanır: consumer tarafı, provider'ı simüle eden mock tabanlı sunucularla test edilir; provider tarafı ise adaptörün bağımlılıklarını taklit eden mock'larla test edilir.

18. Persistence entegrasyon testi

Persistence (kalıcılık) entegrasyon testinin temel amacı, servisin veritabanıyla etkileşimde olduğu durumlarda veritabanıyla yapılan alışverişin doğru gerçekleşip gerçekleşmediğini test etmektir. Kitap bunu dört aşamaya ayırır:

- Setup: veritabanı şeması oluşturulur ve veritabanı test edenin önceden belirlediği bilinen bir duruma getirilir.
- Execute: veritabanı işlemi gerçekleştirilir.
- Verify: veritabanından nesne çekilerek önceden bilinen durumla tutarlılığı test edilir.
- Teardown: değişiklikler geri alınarak veritabanı eski haline getirilir.

Öğrenciye göre kitap teardown'ı isteğe bağlı sayar; setup, execute ve verify persistence entegrasyon testinin üç temel aşamasıdır.

Örnek, şekildeki 3 numaralı etkileşimdir: `OrderRepository` veritabanıyla etkileşim halindedir. Bu testte Order aggregate'ı ve `OrderRepository` için bir kalıcılık entegrasyon testi yapılır ve iki işlem gerçekleştirilir: (1) Order nesnesi oluşturulur ve veritabanına kaydedilir; (2) Order nesnesi veritabanından çekilir ve alanlarının doğruluğuna bakılır (ekrandan, 01:24:15, Listing 10.1):

```
@RunWith(SpringRunner.class)
@SpringBootTest(classes = OrderJpaTestConfiguration.class)
public class OrderJpaTest {

    @Autowired
    private OrderRepository orderRepository;

    @Autowired
    private TransactionTemplate transactionTemplate;

    @Test
    public void shouldSaveAndLoadOrder() {

        Long orderId = transactionTemplate.execute((ts) -> {
            Order order =
                    new Order(CONSUMER_ID, AJANTA_ID, CHICKEN_VINDALOO_LINE_ITEMS);
            orderRepository.save(order);
            return order.getId();
```

```
        });

        transactionTemplate.execute((ts) -> {
            Order order = orderRepository.findById(orderId).get();

            assertEquals(OrderState.APPROVAL_PENDING, order.getState());
            assertEquals(AJANTA_ID, order.getRestaurantId());
            assertEquals(CONSUMER_ID, order.getConsumerId().longValue());
            assertEquals(CHICKEN_VINDALOO_LINE_ITEMS, order.getLineItems());
            return null;
        });
    }
}
```

`shouldSaveAndLoadOrder` metodunda verilen argümanlar test eden tarafından bilinen değerlerdir. Sipariş nesnesi veritabanına kaydedilir, hemen ardından veritabanından çekilerek alanlar karşılaştırılır: durum `APPROVAL_PENDING`, restoran kimliği, müşteri kimliği ve sipariş kalemleri. Alanlar doğruysa test başarıyla geçer.

19. REST request/response için sözleşme testleri ve Spring Cloud Contract

19.1 REST tabanlı request/response etkileşimi

REST tabanlı request/response etkileşiminde bir istemci ve bir sunucu vardır; istemci HTTP istekleri gönderir ve yanıt alır. Doğru HTTP isteğini göndermek ve servisin istemcinin beklediği yanıtı geri göndermesi önemli ölçütlerdir; bunların testi entegrasyon testi kapsamında yapılır. Örnekte API Gateway ile Order Service arasındaki REST tabanlı iletişimin sözleşmelere uygun gerçekleşip gerçekleşmediği test edilir.

19.2 Spring Cloud Contract

Öğrenciye göre entegrasyon testinin merkezinde Spring Cloud Contract vardır. Bu araç mikroservisler arasındaki iletişimi test etmek için kullanılır ve özellikle sözleşme tabanlı iletişimlerde testleri büyük ölçüde kolaylaştırır. Hem provider hem consumer tarafı için sözleşme testlerinin yazılmasını sağlar:

- Consumer tarafı için Spring Cloud Contract bir WireMock sunucusu, yani taklitçi bir sunucu yapılandırır. Provider ayağa kaldırılmadan onu taklit eden bir sunucu kurulur ve test çok daha hızlı yapılır.
- Provider tarafı için Spring Cloud Contract bir kod üreteci gibi çalışır: HTTP test sınıfını oluşturur ve `OrderController`'ı test edecek kodu üretir. Bunun için REST Assured MockMvc kullanır. REST Assured MockMvc, RESTful web servislerinin Spring'in MockMvc altyapısıyla test edilmesini sağlayan bir araçtır.

Öğrenciye göre sonraki etkileşim tiplerinde de Spring Cloud Contract'ın temel rolü aşağı yukarı aynıdır.

19.3 REST API için örnek sözleşme

Sözleşme (contract), bir yazılımda iki sistemin ya da iki servisin nasıl etkileşime gireceğini belirleyen bir anlaşmadır. Hem consumer hem provider bu sözleşmenin içeriğine bağlı kalmak zorundadır; entegrasyon testleri bu bağlılığı kontrol eder. Slayttaki tanım: bir REST API sözleşmesi, bir istemci ile bir sunucu arasındaki etkileşimi tanımlayan bir sözleşmedir; yanıt ve istek içerikleri, HTTP yöntemi, header'lar ve yanıt kodu gibi parametreler burada tanımlanır (ekrandan, 01:27:30, Listing 10.2):

```
org.springframework.cloud.contract.spec.Contract.make {
    request {
        method 'GET'
        url '/orders/1223232'
    }
```

```
    response {
        status 200
        headers {
            header('Content-Type': 'application/json;charset=UTF-8')
        }
        body('''{"orderId" : "1223232", "state" : "APPROVAL_PENDING"}''')
    }
}
```

Sözleşmeye göre `GET /orders/1223232` isteği, `200` durum kodu, JSON içerik tipi ve sipariş kimliği ile `APPROVAL_PENDING` durumunu içeren bir gövdeyle yanıtlanmalıdır. Spring Cloud Contract bu sözleşmeden yararlanarak hem consumer hem provider testlerini üretir.

19.4 Provider tarafı: Order Service için sözleşme testi

Provider tarafındaki test, `OrderController`'ın doğru yapılandırıldığından ve belirtilen mock verilerle doğru yanıt verdiğinden emin olur. Spring Cloud Contract tarafından üretilen testlerin taban sınıfı `HttpBase`, testin kurulumundan sorumludur; amacı, API'nin istemcilerin istediği şekilde yanıt verdiğini doğrulamak için gerekli ortamı hazırlamaktır (ekrandan, 01:28:40, Listing 10.3):

```
public abstract class HttpBase {

    private StandaloneMockMvcBuilder controllers(Object... controllers) {
        ...
        return MockMvcBuilders.standaloneSetup(controllers)
                              .setMessageConverters(...);
    }

    @Before
    public void setup() {
        OrderService orderService = mock(OrderService.class);
        OrderRepository orderRepository = mock(OrderRepository.class);
        OrderController orderController =
                new OrderController(orderService, orderRepository);

        when(orderRepository.findById(1223232L))
                .thenReturn(Optional.of(OrderDetailsMother.CHICKEN_VINDALOO_ORDER));
        ...
        RestAssuredMockMvc.standaloneSetup(controllers(orderController));
    }
}
```

Setup kısmında `OrderService` ve `OrderRepository` nesneleri mock olarak, yani taklitçi nesneler olarak oluşturulur; ardından `OrderController` bunlarla kurulur. `findById` çağrısı sözleşmedeki sipariş kimliğiyle yapıldığında örnek bir siparişin döndürülmesi sağlanır. Son satırda REST Assured MockMvc, Spring MVC'yi `OrderController` ile yapılandırır.

19.5 Consumer tarafı: OrderServiceProxy için entegrasyon testi

API Gateway'in `OrderServiceProxy` sınıfı `GET /orders/{orderId}` uç noktasını çağırır. `OrderServiceProxyIntegrationTest` test sınıfı bu çağrının sözleşmelere uygunluğunu denetler. Sınıf, Spring Cloud Contract'ın sağladığı `@AutoConfigureStubRunner` annotasyonuyla işaretlenmiştir; bu annotasyon Spring Cloud Contract'a WireMock sunucusunu rastgele bir port üzerinde çalıştırmasını ve belirtilen sözleşmelerle yapılandırmasını söyler (ekrandan, 01:30:00, Listing 10.4):

```
@RunWith(SpringRunner.class)
@SpringBootTest(classes = TestConfiguration.class,
        webEnvironment = SpringBootTest.WebEnvironment.NONE)
@AutoConfigureStubRunner(ids =
        {"net.chrisrichardson.ftgo.contracts:ftgo-order-service-contracts"},
```

```
        workOffline = false)
@DirtiesContext
public class OrderServiceProxyIntegrationTest {

    @Value("${stubrunner.runningstubs.ftgo-order-service-contracts.port}")
    private int port;
    private OrderDestinations orderDestinations;
    private OrderServiceProxy orderService;

    @Before
    public void setUp() throws Exception {
        orderDestinations = new OrderDestinations();
        String orderServiceUrl = "http://localhost:" + port;
        orderDestinations.setOrderServiceUrl(orderServiceUrl);
        orderService = new OrderServiceProxy(orderDestinations,
                WebClient.create());
    }

    @Test
    public void shouldVerifyExistingCustomer() {
        OrderInfo result = orderService.findOrderById("1223232").block();
        assertEquals("1223232", result.getOrderId());
        assertEquals("APPROVAL_PENDING", result.getState());
    }

    @Test(expected = OrderNotFoundException.class)
    public void shouldFailToFindMissingOrder() {
        orderService.findOrderById("555").block();
    }
}
```

Setup kısmı testlerden hemen önce çalışır: `OrderDestinations` nesnesi oluşturulur ve `OrderServiceProxy`'nin doğru URL ile yapılandırılması sağlanır. Order Service URL'i, WireMock'un çalıştığı port kullanılarak dinamik olarak belirlenir. Testi yapan iki metot vardır:

- `shouldVerifyExistingCustomer`: `OrderServiceProxy` ile `findOrderById("1223232")` sorgulaması yapılır ve siparişin kimliğinin verilen kimlik, durumunun `APPROVAL_PENDING` olduğu doğrulanır. İkisi de doğrulanırsa test geçer.
- `shouldFailToFindMissingOrder`: olmayan bir siparişin sorgulaması test edilir. "555" kimlikli bir sipariş yoktur; beklenen davranış `OrderNotFoundException` fırlatılmasıdır.

20. Publish/subscribe ve asenkron request/response testleri

20.1 Publish/subscribe etkileşimi

Order Service, siparişler güncellendikçe ve yeni siparişler geldikçe event yayınlar. Bu event'lere abone olan taraf Order History Service'tir; event'in consumer'larından biridir. Bu etkileşimin doğru gerçekleşip gerçekleşmediğinin testi de yapılması gereken bir entegrasyon testidir.

- Provider tarafında `OrderDomainEventPublisher` sınıfının sözleşmeye uygun event yayınlayıp yayınlamadığı doğrulanır.
- Consumer tarafında Order History event handler'ının sözleşmedeki örnek event'leri doğru tükettiği doğrulanır. Burada bir mesajlaşma stub'ı vardır: Spring Cloud Contract tarafından oluşturulmuş taklitçi bir provider, ve bu provider'ın yayınladığı event'lerin doğru tüketilip tüketilmediği consumer tarafında test edilir.

20.2 Publish/subscribe için örnek sözleşme

Slayttaki sözleşme `OrderCreated` event'ini tanımlar; event'in kanalı, gövdesi ve mesaj başlıkları gibi bilgileri içerir (ekrandan, 01:32:15, Listing 10.5):

```
package contracts;

org.springframework.cloud.contract.spec.Contract.make {
    label 'orderCreatedEvent'
    input {
        triggeredBy('orderCreated()')
    }

    outputMessage {
        sentTo('net.chrisrichardson.ftgo.orderservice.domain.Order')
        body('''{"orderDetails":{"lineItems":[{"quantity":5,"menuItemId":"1",
                 "name":"Chicken Vindaloo","price":"12.34","total":"61.70"}],
                 "orderTotal":"61.70","restaurantId":1,
        "consumerId":1511300065921},"orderState":"APPROVAL_PENDING"}''')
        headers {
            header('event-aggregate-type',
                        'net.chrisrichardson.ftgo.orderservice.domain.Order')
            header('event-aggregate-id', '1')
        }
    }
}
```

- `label`: consumer testi tarafından, Spring Cloud Contract'ın event'i yayınlamasını tetiklemek için kullanılır.
- `input` içindeki `triggeredBy`: kod üretimiyle oluşturulan provider testinin, event'in yayınlanmasını tetiklemek için çağırdığı taban sınıf metodunun adıdır (`orderCreated()`).
- `outputMessage`: yayınlanan `OrderCreated` domain event'idir; gönderildiği kanal (`sentTo`), gövdesi ve başlıkları (`event-aggregate-type`, `event-aggregate-id`) tanımlanır.

Gövdedeki sayılar tutarlıdır: tek kalemde 5 adet, birim fiyat 12.34, kalem toplamı $5 \times 12.34 = 61.70$ ve sipariş toplamı da 61.70'tir.

Provider tarafı testler için Spring Cloud Contract'ın kullandığı soyut taban sınıf `MessagingBase` sınıfıdır. Bu sınıf `OrderDomainEventPublisher`'ı yapılandırır; testin temel amacı provider tarafında event'lerin doğru yayınlandığını doğrulamaktır. Spring Cloud Contract sayesinde bu testler de sözleşmelere dayalı yazılır ve etkileşimlerin doğru gerçekleştiğinden emin olunur.

20.3 Asenkron request/response etkileşimi

Son etkileşim tipi asenkron request/response ilişkisidir. Sözleşme yazımı REST request/response'a benzer. İki taraf vardır: requester (istek gönderen, komut mesajını gönderen servis) ve replier (yanıtlayıcı, komut mesajını işleyip yanıt gönderen servis). İki tarafın komut mesajının kanal adı ve komut ile yanıt mesajlarının yapısı üzerinde anlaşması gerekir; bu anlaşmaya uyulup uyulmadığı da entegrasyon testlerinde denetlenir.

- Provider tarafında Kitchen Service command handler'ının komutları doğru işlediği ve yanıtları doğru gönderdiği doğrulanır.
- Consumer tarafında `KitchenServiceProxy`'nin sözleşmelere uygun komut gönderdiği ve sözleşmedeki örnek yanıtları doğru işlediği doğrulanır.

Böylece iki taraf için de etkileşimin doğru gerçekleştiği doğrulanmış olur. Öğrenci sunumu entegrasyon testinin ne olduğu ve niçin kullanıldığı, farklı etkileşim tipleri için entegrasyon testlerinin nasıl işlediği ve Spring Cloud Contract'ın ne olduğu ve niçin kullanıldığı sorularıyla özetler.

Örnek Sorular

Bu derste ekranda quiz ya da alıştırma gösterilmedi. Aşağıdaki sorular sunumlarda sözlü olarak sorulan ve sunum içinde cevaplanan sorulardır.

Soru 1 (derste sözlü, 00:37:33): Mikroservis mimarisinde monolitte tek bir çağrıyla alınan bilgi için dört ayrı API çağrısı gerekiyorsa istemci dört farklı URL'i mi bilmek zorundadır, servislerle nasıl iletişim kuracaktır?

Cevap (derste verildi): Hayır. Araya bir API Gateway konur. Gateway dışarıdan gelen bütün çağrılara tek bir giriş noktası sunar, istekleri doğru servise yönlendirir ve gerekirse birden çok servisin yanıtını birleştirip istemciye tek yanıt döner. İstemci arka uç servislerine doğrudan maruz kalmaz.

Soru 2 (derste sözlü, 00:34:13): API Gateway gerçekleştirilirken hazır bir ürün mü kullanılmalı, yoksa kendimiz mi yazmalıyız?

Cevap (derste verildi): Hazır ürünlerin her birinin eksiği vardır: AWS API Gateway API composition sunmaz, AWS Application Load Balancer çok sınırlı gateway özelliğine sahiptir, Kong/Traefik esnektir ama operasyonel çaba ister. Kendi gateway'ini yazmak için yönlendirme kurallarını ve HTTP proxy davranışını (header'lar dahil) doğru uygulamak yeterlidir; öğrenciye göre kitap bunun için Spring Cloud Gateway çatısını önerir.

Soru 3 (derste sözlü, 01:18:47): Entegrasyon testini neden yapıyoruz, neden önemlidir?

Cevap (derste verildi): Servisler arasındaki iletişimin doğru gerçekleştiğini ve sözleşmelere uyulduğunu doğrulamak için. Beklenmeyen davranışları uçtan uca testlerden önce yakalamak maliyet ve zaman kazandırır; ayrıca geliştiriciler arası işbirliğini kolaylaştırır, performans analizine imkan verir ve regresyonları önler.

Soru 4 (derste sözlü, 01:16:23): `UserService.getUserName(int userId)` fonksiyonu için solitary birim testi yazılırken kaç test yazılmalıdır ve neden?

Cevap (derste verildi): İki test. Fonksiyonun iki durumu vardır: repository `null` döndürürse "User not found", bir kullanıcı döndürürse kullanıcının adı döner. `UserRepository` mock'lanarak her iki dal için birer test yazılır; böylece fonksiyonun tamamı kapsanır.

Dersin Toparlaması

Bu derste altı öğrenci sunumuyla mikroservis kitabının sorgu, dış API ve test bölümleri işlendi. CQRS, birden çok servise yayılan sorguların API composition ile pahalı ve verimsiz hale gelmesine karşı, komut tarafının yayınladığı event'lerle güncellenen ayrı bir sorgu modeli ve veritabanı önerir; bedeli ek karmaşıklık ve replikasyon gecikmesidir. API Gateway, istemcilere tek bir giriş noktası sunarak yönlendirme, API composition, protokol çevirisi ve kimlik doğrulama gibi edge function'ları üstlenir; tek hata noktası olma riskine karşı BFF deseniyle istemci türüne göre bölünebilir. Gerçekleştirmede hazır ürünlerin eksikleri ve kendi gateway'ini yazmanın kritik noktaları tartışıldı.

Test sunumları ise test piramidini temel aldı: çok sayıda hızlı birim testi, daha az entegrasyon ve bileşen testi, en az uçtan uca test. Birim testlerinde solitary (bağımlılıklar mock'lanır) ve sociable (gerçek bağımlılıklarla) ayrımı, AAA deseni ve stub/mock kavramları örneklerle gösterildi. Entegrasyon testlerinde servisin tamamı yerine iletişimi gerçekleştiren adaptör sınıfları test edilir; Order Service'in REST, publish/subscribe, kalıcılık ve asenkron request/response etkileşimleri için Spring Cloud Contract ile sözleşmelerden hem consumer hem provider testleri üretilir. CQRS'in tasarımıyla ilgili sunum sonraki haftaya kaldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.